

作家风格识别

1.实验介绍

1.1实验背景

作家风格是作家在作品中表现出来的独特的审美风貌。

通过分析作品的写作风格来识别作者这一研究有很多应用，比如可以帮助人们鉴定某些存在争议的文学作品的作者、判断文章是否剽窃他人作品等。

作者识别其实就是一个文本分类的过程，文本分类就是在给定的分类体系下，根据文本的内容自动地确定文本所关联的类别。

写作风格学就是通过统计的方法来分析作者的写作风格，作者的写作风格是其在语言文字表达活动中的个人言语特征，是人格在语言活动中的某种体现。

1.2 实验要求

- 建立深度神经网络模型，对一段文本信息进行检测识别出该文本对应的作者。
- 绘制深度神经网络模型图、绘制并分析学习曲线。
- 用准确率等指标对模型进行评估。

1.3 实验环境

可以使用基于 Python 分词库进行文本分词处理，使用 Numpy 库进行相关数值运算，使用 Keras 等框架建立深度学习模型等。

1.4 参考资料

jieba: <https://github.com/fxsjy/jieba>

Numpy: <https://www.numpy.org/>

Pytorch: <https://pytorch.org/docs/stable/index.html>

TorchText: <https://torchtext.readthedocs.io/en/latest/>

2.实验内容

2.1 介绍数据集

该数据集包含了 8438 个经典中国文学作品片段，对应文件分别以作家姓名的首字母大写命名。

数据集中的作品片段分别取自 5 位作家的经典作品，分别是：

序号	中文名	英文名	文本片段个数
1	鲁迅	LX	1500 条
2	莫言	MY	2219 条
3	钱钟书	QZS	1419 条
4	王小波	WXB	1300 条
5	张爱玲	ZAL	2000 条

- 其中截取的片段长度在 100~200 个中文字符不等
- 数据集路径为 `dataset/` 以作者名字首字母缩写命名

2.2 数据集预处理

在做文本挖掘的时候，首先要做的预处理就是分词。
英文单词天然有空格隔开容易按照空格分词，但是也有时候需要把多个单词做为一个分词，比如一些名词如 "New York"，需要做为一个词看待。
而中文由于没有空格，分词就是一个需要专门去解决的问题了。
这里我们使用 jieba 包进行分词，使用**精确模式**、**全模式**和**搜索引擎模式**进行分词对比。

使用 TF-IDF 算法统计各个作品的关键词频率

TF-IDF (term frequency-inverse document frequency, 词频-逆向文件频率) 是一种用于信息检索与文本挖掘的常用加权技术。

- TF-IDF是一种统计方法，用以评估一字词对于一个文件集或一个语料库中的其中一份文件的重要程度。
字词的重要性随着它在文件中出现的次数成正比增加，但同时会随着它在语料库中出现的频率成反比下降。
- TF-IDF的主要思想是：如果某个单词在一篇文章中出现的频率TF高，并且在其他文章中很少出现，则认为此词或者短语具有很好的类别区分能力，适合用来分类。
这里我们使用 jieba 中的默认语料库来进行关键词抽取，并展示每位作者前 5 个关键词

2.3 模型调整

在训练之前，需要先将数据集划分为训练集和验证集，并且确定每次训练批次的大小，验证集所占比例 `valid_split` 我尝试了 0.1-0.3 的取值，最终选择了效果较好的 0.2——验证集比例过大容易出现过拟合现象，过小时模型验证不准确，也不利于训练。训练批次 `batch` 的大小我尝试了 16、32、64，在测试中 64 的表现最好。

刚开始，我尝试了三个隐含层 512, 1024, 1024 的神经网络，在 `epochs=30`，`Adam`，`nn.CrossEntropyLoss()` 的情况下，训练集准确率可以稳定保持在 1 而验证集只有 0.8+，说明模型有一定程度的过拟合，泛化能力不强，测试结果 50 个文本只识别正确 42 个。

随后我尝试了删掉一层网络，只保留 512, 1024 两个隐含层再进行测试，测试结果有显著提高。在尝试对网络深度和宽度的调整后，最终我选择了一个隐含层 (700) 的神经网络，得到了较好的结果。

最优结果

经过多种尝试组合，最终当神经网络有一个 700 的隐含层，当 `valid_split = 0.2`，`epoch=100`，`batch=64`，`topK=500`，`torch.manual_seed(10)` 时，得到了最优结果，训练时的准确率为 1，测试结果可以正确识别 49 个。

测试详情

X

测试点	状态	时长	结果
		3s	测试完成 一共50个文本，预测正确49个

3. 实验心得

经过本实验我了解了运用神经网络进行作家风格识别的基本思想，这是一个多分类问题，大致思想是利用 Python 分词库进行文本分词处理，然后用 TF-IDF 算法统计词频并筛选，得到不同作者的特征，利用这些特征训练模型，从而得到可以识别文风的模型。

附录

训练代码

```
import os
import numpy as np
import jieba as jb
import jieba.analyse
import torch
import torch.nn as nn
from torch.utils import data
import matplotlib.pyplot as plt

device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

int2author = ['LX', 'MY', 'QZS', 'WXB', 'ZAL']
author_num = len(int2author)
author2int = {author: i for i, author in enumerate(int2author)}

# dataset = {(sentence, label), }
dataset_init = []
path = 'dataset/'
for file in os.listdir(path):
    if not os.path.isdir(file) and not file[0] == '.': # 跳过隐藏文件和文件夹
        with open(os.path.join(path, file), 'r', encoding='UTF-8') as f: # 打开文件
            for line in f.readlines():
                dataset_init.append((line, author2int[file[:-4]]))

# 将片段组合在一起后进行词频统计
str_full = ['' for _ in range(author_num)]
for sentence, label in dataset_init:
    str_full[label] += sentence

# 词频特征统计，取出各个作家前 500 的词
words = set()
for label, text in enumerate(str_full):
    for word in jb.analyse.extract_tags(text, topK=500, withweight=False):
        words.add(word)

int2word = list(words)
word_num = len(int2word)
word2int = {word: i for i, word in enumerate(int2word)}

features = torch.zeros((len(dataset_init), word_num))
labels = torch.zeros(len(dataset_init))
for i, (sentence, author_idx) in enumerate(dataset_init):
    feature = torch.zeros(word_num, dtype=torch.float)
    for word in jb.lcut(sentence):
        if word in words:
```

```

        feature[word2int[word]] += 1
    if feature.sum():
        feature /= feature.sum()
        features[i] = feature
        labels[i] = author_idx
    else:
        labels[i] = 5 # 表示识别不了作者

dataset = data.TensorDataset(features, labels)

# 划分数据集
torch.manual_seed(10) # 设置随机种子
valid_split = 0.2
train_size = int((1 - valid_split) * len(dataset))
valid_size = len(dataset) - train_size
train_dataset, test_dataset = torch.utils.data.random_split(dataset,
    [train_size, valid_size])
# 创建一个 DataLoader 对象
train_loader = data.DataLoader(train_dataset, batch_size=64, shuffle=True)
valid_loader = data.DataLoader(test_dataset, batch_size=2500, shuffle=True)

model = nn.Sequential(
    nn.Linear(word_num, 700),
    nn.ReLU(),
    nn.Linear(700, 6),
).to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
best_acc = 0
best_model = model.cpu().state_dict().copy()
train_acc_list = []
valid_acc_list = []

for epoch in range(100):
    for step, (b_x, b_y) in enumerate(train_loader):
        b_x = b_x.to(device)
        b_y = b_y.to(device)
        out = model(b_x)
        loss = loss_fn(out, b_y.long())
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        train_acc = np.mean((torch.argmax(out, 1) == b_y).cpu().numpy())

    with torch.no_grad():
        for b_x, b_y in valid_loader:
            b_x = b_x.to(device)
            b_y = b_y.to(device)
            out = model(b_x)
            valid_acc = np.mean((torch.argmax(out, 1) == b_y).cpu().numpy())
        if valid_acc > best_acc:
            best_acc = valid_acc
            best_model = model.cpu().state_dict().copy()
    print('epoch:%d | valid_acc:%.4f | train_acc:%.4f' %
        (epoch, valid_acc, train_acc))
    # 记录每个 epoch 结束后的训练准确度和验证准确度

```

```

        train_acc_list.append(train_acc)
        valid_acc_list.append(valid_acc)
        if train_acc==1 and valid_acc==1:
            break
# 绘制学习曲线
epochs = range(1, 101)
plt.plot(epochs, train_acc_list, 'b', label='Training accuracy')
plt.plot(epochs, valid_acc_list, 'r', label='Validation accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
# 保存图片
plt.savefig('results/learning_curve_100.png') # 图片保存路径和名称, 可以根据需要更改格式

print('best accuracy:%.4f' % (best_acc, ))
torch.save({
    'word2int': word2int,
    'int2author': int2author,
    'model': best_model,
}, 'results/nn_model_100.pth')

```

main.py

```

# ===== 提交 Notebook 训练模型结果数据处理参考示范 =====
# 导入相关包
import copy
import os
import random
import numpy as np
import jieba as jb
import jieba.analyse
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.nn import init
import torch.nn.functional as f
from torchtext import data
from torchtext import datasets
from torchtext.data import Field
from torchtext.data import Dataset
from torchtext.data import Iterator
from torchtext.data import Example
from torchtext.data import BucketIterator

# -----
# 本 cell 代码仅为 Notebook 训练模型结果进行平台测试代码示范
# 可以实现个人数据处理的方式, 平台测试通过即可提交代码
# -----

import torch
import torch.nn as nn
import jieba as jb

device = torch.device('cuda:0' if torch.cuda.is_available() else 'cpu')

```

```
config_path = 'results/nn_model_100_49.pth'
config = torch.load(config_path)

word2int = config['word2int']
int2author = config['int2author']
word_num = len(word2int)

model = nn.Sequential(
    nn.Linear(word_num, 700),
    nn.ReLU(),
    nn.Linear(700, 6),
)
model.load_state_dict(config['model'])
int2author.append(int2author[0])

def predict(text):
    feature = torch.zeros(word_num)
    for word in jb.lcut(text):
        if word in word2int:
            feature[word2int[word]] += 1
    feature /= feature.sum()
    model.eval()
    out = model(feature.unsqueeze(dim=0))
    pred = torch.argmax(out, 1)[0]
    return int2author[pred]
```